Table des matières

Cours 04 — Bâtir l'étoile des ventes 1
Objectifs 2
1. De la vue au produit 2
2. Le premier paquet 3
3. La clé de substitution, fabriquée 3
4. `dim_date`, la dimension que personne ne livre 4
5. Des vues de staging aux tables de dimension 5
6. La couche `INTERMEDIATE`, et le fait mince 5
7. Le fait des ventes 6
8. Réconcilier, puis écrire ce qu'on livre 6
9. Ce qu'une table coûte, ici et ailleurs 7
Points clés à retenir 8
Exercices 8

Cours 04 — Bâtir l'étoile des ventes

Théorie · Module 4/9 — Formation Analytics Engineering · intermédiaire

Où vous êtes : `RAW` → `STAGING` → **`INTERMEDIATE`** → **`MARTS`**. Le Cours 04 bâtit la première étoile, celle que le Cours 02 a conçue sur papier et prototypée à la main.

En 30 secondes :

- Une couche `MARTS` n'est pas une couche de plus : c'est le produit, celui que d'autres équipes liront sans vous demander comment il est fait.
- Un paquet dbt apporte des macros que vous n'écrivez pas. `dbt_utils` est le seul de la formation, et il donne la table de dates et la clé de substitution.
- La clé de substitution se fabrique par hachage de la clé naturelle. Comme le hachage est déterministe, le fait la recalcule sans joindre la dimension.
- La couche `INTERMEDIATE` porte les jointures et l'enrichissement pour que le fait reste mince. En `ephemeral`, elle n'existe nulle part dans la base.
- Une étoile qui ne se réconcilie pas avec sa source ne se livre pas. Chez Boréal, le témoin est la valeur nette des en-têtes de commande, et il doit tomber au cent près.

© 2026 iAlexMG. Tous droits réservés.

L'idée du cours. Vous avez déjà écrit cette étoile une fois, au Labo 02, dans un schéma `prototype` que le Cours 03 vous a fait jeter. Ce cours la refait, et rien de la conception ne change : même grain, mêmes dimensions, mêmes chiffres. Ce qui change, c'est que le résultat est versionné, rejouable et relié, et que vous ne pouvez plus vous tromper d'ordre d'exécution même en le voulant. À la fin du labo, la valeur nette de votre fait égalera au cent près celle que l'ERP porte dans ses en-têtes, 430 148 012,16 $.

Objectifs

- Dire ce qui sépare une vue de staging d'une table de la couche `MARTS`.
- Installer un paquet dbt, et dire ce que le projet en garde dans son dépôt.
- Fabriquer une clé de substitution par macro, et expliquer pourquoi le fait n'a pas à joindre la dimension pour la retrouver.
- Bâtir une dimension de dates par `date_spine`, avec ses libellés en français.
- Tirer une dimension d'une vue de staging, et nommer ce que le type 1 fait perdre.
- Dire ce qu'on met dans la couche `INTERMEDIATE`, et ce qu'un modèle `ephemeral` coûte.
- Écrire un fait au grain déclaré, et le vérifier.
- Réconcilier un fait avec sa source, et déclarer ce qu'on livre.

---

1. De la vue au produit

Les trois vues du Cours 03 rendent la `RAW` lisible. Personne ne s'en sert pour répondre à une question de métier, et ce n'est pas leur rôle : elles n'ont ni grain déclaré, ni clé fabriquée, ni mesure.

La couche `MARTS` est le premier endroit du projet qui s'adresse à quelqu'un d'autre. Trois choses y changent d'un coup.

	STAGING	**MARTS**
Qui le lit	vos propres modèles	les analystes, les rapports, les autres formations
Ce qui le structure	la source, une vue par table livrée	la conception, le grain déclaré au Cours 02
La matérialisation	des vues	des tables
Ce qui le fait changer	un changement de la source	une décision de modélisation

© 2026 iAlexMG. Tous droits réservés.

La troisième ligne suit des deux premières. Une vue rejoue sa requête à chaque lecture, ce qui est sans conséquence sur un renommage ; un fait de 75 652 lignes qui joint trois dimensions et qu'un rapport interroge vingt fois par jour se calcule une fois, pas vingt.

La quatrième est la plus importante à long terme. Une vue de staging suit sa source sans discuter ; une table des `MARTS` porte des décisions, et c'est pour cela qu'elle se relit, se teste et se documente. Le Cours 05 s'occupera des deux derniers points.

Labo : `Labo_04_AE.md`, étape 3.

2. Le premier paquet

Un paquet dbt est une bibliothèque de macros, installée dans le projet et appelée comme si vous l'aviez écrite. Trois fichiers gouvernent l'affaire.

Fichier	Ce qu'il fait
`packages.yml`	déclare ce que le projet veut, et dans quelle fourchette de versions
`package-lock.yml`	garde la version exactement résolue, pour que deux postes installent la même
`dbt_packages/`	reçoit le code téléchargé, et ne se versionne jamais

La commande qui lit le premier et remplit le troisième est `dbt deps`. Elle ne se lance pas toute seule : un projet qui déclare un paquet sans l'installer s'arrête à la première commande, avec un message qui dit quoi faire (« Run "dbt deps" to install package dependencies »). C'est un bon message, et ils ne le sont pas tous.

La formation n'installe qu'un paquet, `dbt_utils`, et lui en prend deux macros : `date_spine`, qui fabrique une suite de dates, et `generate_surrogate_key`, qui fabrique une clé. La fourchette déclarée est `[">=1.0.0", "<2.0.0"]`, et elle se résout aujourd'hui en 1.4.1.

Piège — sous Windows, `dbt deps` échoue sur un chemin trop long, et le message ne dit pas pourquoi. Le plus long fichier de `dbt_utils` ajoute 127 caractères au chemin de votre projet, et Windows refuse d'écrire au-delà de 260 tant que l'option des chemins longs n'est pas activée. L'erreur est un `FileNotFoundError` qui nomme un fichier de test du paquet, ce qui n'aide personne. La parade est de ranger `Projet_Boreal` à un chemin court ; un lecteur virtuel posé par `subst` ne contourne rien, Windows résolvant toujours le chemin réel.

Labo : `Labo_04_AE.md`, étape 1.

3. La clé de substitution, fabriquée

Le Cours 02 a dit pourquoi une dimension a deux clés. Voici comment on fabrique la seconde.

`{{ dbt_utils.generate_surrogate_key(['code_concessionnaire']) }}` produit un hachage de la clé naturelle. Le SQL compilé tient en une ligne, et il vaut la peine de la regarder : `md5(cast(coalesce(cast(code_concessionnaire as TEXT), '_dbt_utils_surrogate_key_null_') as TEXT))`. Sur une colonne non vide, c'est le `md5` de sa valeur, exactement celui que vous obtiendriez en l'écrivant à la main.

© 2026 iAlexMG. Tous droits réservés.

Trois propriétés en découlent, et la troisième est celle qui change la façon d'écrire un fait.

La propriété	Ce qu'elle permet
Déterministe	la même clé naturelle donne toujours le même hachage, dans tous les modèles
Uniforme	une chaîne de 32 caractères, quelle que soit la forme de la clé source
Calculable sans la dimension	le fait recalcule la clé au lieu de joindre pour aller la chercher

C'est cette troisième propriété qui évite une jointure par dimension dans le fait. Le fait ne demande pas à `dim_concessionnaire` quelle est la clé du concessionnaire `0000200077` : il la recalcule, et elle tombe juste par construction.

Le prix à payer se dit en une phrase : rien ne garantit alors que la ligne existe dans la dimension. Un fait peut porter la clé d'un concessionnaire qu'aucune dimension ne décrit, et la jointure du rapport perdra silencieusement ces lignes. C'est exactement ce que le test `relationships` du Cours 05 vérifie, et c'est pour cela qu'il n'est pas facultatif.

Piège — l'ordre des colonnes compte, et les valeurs vides se confondent. Une clé fabriquée sur deux colonnes dans un ordre donne un hachage, et dans l'autre ordre un hachage différent : `md5('AB')` vaut `b86fc6b051f63d73…` et `md5('BA')` vaut `5fc810cf62601df8…`. Et comme la macro remplace une valeur vide par un texte sentinelle, deux lignes dont la clé naturelle est vide reçoivent la même clé de substitution. Une clé se fabrique sur des colonnes obligatoires, dans un ordre écrit une fois.

Labo : `Labo_04_AE.md`, étapes 5 et 8.

4. `dim_date`, la dimension que personne ne livre

Toutes les autres dimensions se tirent d'une source. Celle des dates se fabrique, et c'est la seule table de l'entrepôt qui ne doit rien à l'ERP.

La macro `dbt_utils.date_spine` produit une ligne par jour entre deux bornes, dans une colonne nommée `date_day`. La borne de fin est exclue : demander `cast('2026-04-01' as date)` donne un dernier jour au 31 mars, et c'est ainsi qu'on obtient les 1 551 jours du 2022-01-01 au 2026-03-31.

Deux questions se posent devant cette table, et elles ont la même réponse.

La première : pourquoi aller jusqu'en mars 2026 quand les ventes s'arrêtent au 31 décembre 2025. Une dimension de dates couvre un calendrier, pas des faits. Chez Boréal, votre fait ne touche que 1 012 jours sur les 1 551 ; les autres sont des jours sans vente, et une table qui les porte permet de dire « zéro » là où une jointure sur les seules dates vendues ne dit rien du tout. Les mois de 2026 arriveront au Cours 07, et la dimension les attend déjà.

La seconde : pourquoi sa clé n'est pas un hachage. Une date est stable, unique, lisible et ne change jamais de valeur : elle se suffit comme clé. L'usage veut souvent un entier de la forme `20250131`, qui rend le même service. Ce qui serait fautif, c'est d'en hacher une : on perdrait la lisibilité et le tri sans rien gagner.

© 2026 iAlexMG. Tous droits réservés.

Reste le piège que le Labo 02 vous a fait constater, et qui se règle ici pour de bon. Sous DuckDB, `strftime(date, '%B')` rend « January » et `dayname()` rend « Monday », sans option de langue. Un entrepôt francophone qui s'appuie dessus livre des mois en anglais à tous ses rapports, et le défaut se corrige ensuite dans chaque rapport, un par un. Le décodage se fait donc une seule fois, par une liste indexée, à l'endroit unique qui fabrique la dimension.

À retenir — un libellé se décode là où il naît. La règle vaut pour les mois comme pour les codes SAP : le jour où quelqu'un répare un libellé ailleurs que dans le modèle qui le produit, il y a deux vérités dans l'entrepôt.

Labo : `Labo_04_AE.md`, étape 4.

5. Des vues de staging aux tables de dimension

Une dimension est une vue de staging à laquelle on ajoute une clé fabriquée et dont on retire ce que personne n'analyse. Le travail est court, et c'est normal : le gros du décodage a été fait au Cours 03.

Ce qu'une dimension ajoute quand même :

- Sa clé de substitution, qui devient la seule que les faits portent.
- Les libellés qu'un analyste veut voir dans un `where`, même s'ils viennent d'une autre table. Une dimension a le droit d'être dénormalisée, c'est toute la différence entre l'étoile et le flocon du Cours 02. La province du concessionnaire, par exemple, est un code de trois lettres qui a son libellé ailleurs dans l'ERP, et rien n'interdit de le ramener ici, à condition de filtrer la langue comme le Cours 02 l'a montré sur les désignations d'articles.

Ce qu'elle ne fait pas encore, dans ce cours : garder l'histoire. Les dimensions du Labo 04 sont de type 1, elles n'ont qu'une ligne par entité, et elles écrasent. La conséquence se mesure, et elle n'est pas cosmétique : chez Boréal, 49 concessionnaires ont changé de palier le 1er juillet 2024, et ces 49 comptent pour 90 314 267,26 $ de ventes antérieures à cette date, soit 35,5 pour cent de la valeur vendue avant la révision. Tant que la dimension reste en type 1, cette valeur est attribuée à des paliers que les concessionnaires n'avaient pas à l'époque.

Ce n'est pas une raison de ne pas livrer l'étoile maintenant. C'est une raison de savoir ce qu'on livre, et le Cours 05 branchera le type 2.

Labo : `Labo_04_AE.md`, étapes 5 et 6.

6. La couche `INTERMEDIATE`, et le fait mince

Entre le staging et le fait, il y a du travail qui n'est ni l'un ni l'autre : joindre les postes à leurs en-têtes, écarter les postes refusés, calculer une marge. Ce travail va dans une couche `INTERMEDIATE`, pour une raison et une seule : que le modèle du fait reste lisible.

Un fait mince est un fait qui ne contient que des clés et des mesures. Quand la jointure, le filtre et le calcul métier sont ailleurs, `fct_ventes` tient en douze lignes et se relit d'un coup d'œil. Quand tout est dedans, il en fait quarante et personne ne sait plus ce qui vient d'où.

© 2026 iAlexMG. Tous droits réservés.

La marge, justement, mérite un mot. C'est une soustraction, donc une mesure additive : la somme des marges de dix lignes est la marge de ces dix lignes, et elle se stocke sans danger. Le taux de marge, lui, ne se stocke pas, et le Cours 02 a dit pourquoi. On garde le numérateur et le dénominateur, le ratio se calcule à la lecture.

Reste la matérialisation. Un modèle intermédiaire lu par un seul autre modèle n'a pas besoin d'exister dans la base, et `ephemeral` le dit à dbt : le SQL est injecté dans le modèle qui l'appelle, sous la forme d'une expression de table commune. Après `dbt run`, le schéma `intermediate` n'existe pas, et interroger `intermediate.int_lignes_enrichies` rend une erreur de catalogue. Le modèle est pourtant bien dans le graphe, et `dbt ls` le liste comme les autres.

Piège — une erreur dans un modèle `ephemeral` est signalée dans le modèle qui l'appelle. Une colonne mal nommée dans `int_lignes_enrichies` sort en « Runtime Error in model fct_ventes », avec un numéro de ligne qui est celui du SQL compilé, pas celui de votre fichier. Le réflexe est d'ouvrir le compilé, dont dbt donne le chemin dans le message, et d'y retrouver la ligne fautive.

Labo : `Labo_04_AE.md`, étape 7.

7. Le fait des ventes

Le grain est déclaré depuis le Cours 02 : une ligne représente un poste d'une commande d'un concessionnaire. Tout le reste en découle.

Les colonnes d'un fait se rangent en trois familles, et savoir laquelle est laquelle évite des erreurs de lecture.

La famille	Chez Boréal
Les clés vers les dimensions	`cle_date`, `cle_concessionnaire`, `cle_produit`
Les mesures	`quantite`, `valeur_nette`, `cout`, `marge`
Les dimensions dégénérées	`numero_commande`, `numero_poste`

La troisième famille est celle du Cours 02 : un numéro de commande est un axe d'analyse légitime, mais il n'a aucun attribut, donc il reste dans le fait plutôt que de justifier une table à une colonne.

Une règle de métier s'applique ici, et elle doit être écrite quelque part de visible : le fait des ventes ne compte pas les postes refusés. Chez Boréal, 2 080 postes sur 77 732 portent un motif de refus, pour 18 001 551,21 $. Les garder gonflerait le chiffre d'affaires de 4,18 pour cent, sur un total parfaitement plausible que personne ne recompterait.

Et un contrôle s'applique à tout fait que vous écrirez : le nombre de lignes doit égaler le nombre de couples distincts qui définissent le grain. S'ils diffèrent, une jointure a dupliqué des lignes, et le fait ne dit plus ce que son grain annonce.

Labo : `Labo_04_AE.md`, étape 8.

8. Réconcilier, puis écrire ce qu'on livre

© 2026 iAlexMG. Tous droits réservés.

Un fait se construit à partir de dizaines de décisions : quel état courant, quelle jointure, quel filtre. Chacune peut être fausse sans rien casser, et un total faux de cinq pour cent se lit comme un total juste. La seule parade est un témoin indépendant.

Un bon témoin ne vient pas du même chemin que ce qu'il contrôle. Chez Boréal, il existe : l'ERP porte, sur chaque en-tête de commande, une valeur nette qu'il a calculée lui-même comme la somme de ses postes non refusés. Votre fait part des postes, le témoin part des en-têtes, et les deux doivent se rejoindre au cent près. C'est le seul chiffre de ce cours qui n'admet aucun écart.

Ce qu'on compare	Valeur
Somme de `valeur_nette` dans `fct_ventes`	430 148 012,16 $
Somme de `valeur_nette` dans `stg_commandes`	430 148 012,16 $
Écart	0,00 $

Deux autres écarts, eux, sont attendus, et le travail consiste à savoir les expliquer avant que quelqu'un ne les découvre dans un rapport. Le fait porte 75 652 lignes et non les 77 732 postes de la source, parce que 2 080 sont refusés. Il porte 25 256 commandes et non les 25 483 de l'ERP, parce que 227 commandes n'ont que des postes refusés et disparaissent donc entièrement.

Il reste un geste, et c'est celui qui fait la différence entre un entrepôt et un dossier de modèles. Ce que vous livrez se déclare : les modèles, leur grain, leurs comptes attendus, et la façon de les reconstruire. Quatre formations en aval liront cette déclaration pour savoir ce qu'elles reçoivent, exactement comme vous avez lu celle de Data Engineering au Cours 01. Le document s'appelle `LIVRAISON.md`, il commence ici, et il se complète jusqu'au capstone.

Labo : `Labo_04_AE.md`, étapes 9 et 10.

9. Ce qu'une table coûte, ici et ailleurs

Sous DuckDB, une table est un objet dans le fichier de la base, reconstruit entièrement à chaque `dbt run`. C'est brutal et c'est bon marché : les neuf modèles du projet, dont le fait de 75 652 lignes, se rebâtissent en moins d'une seconde. La question du coût ne se pose donc pas ici, et c'est précisément pourquoi il faut savoir où elle se pose.

Sur une autre plateforme. Sur Snowflake, une table matérialisée occupe du stockage facturé et sa reconstruction allume un entrepôt virtuel, donc se paie au temps de calcul ; les données y sont découpées en micro-partitions, et une table reconstruite d'un bloc les réécrit toutes. Sur Databricks, la table est un ensemble de fichiers Delta posés sur le stockage objet, qui range des fichiers entiers désignés par une clé, et la reconstruction se paie au temps de grappe. Dans les deux cas, la parade est la même et elle a son cours : la matérialisation `incremental`, qui n'écrit que ce qui a changé, au Cours 07.

Ce qui ne change pas d'une plateforme à l'autre, c'est la raison de matérialiser : un objet lu souvent et par plusieurs se calcule une fois. Le staging reste en vues, les `MARTS` en tables, et ce choix se défend partout de la même façon.

© 2026 iAlexMG. Tous droits réservés.

Recoupements. Les fonctions de fenêtre sont au Cours 04 de la formation SQL avancé et l'agrégation avancée au Cours 06 ; ici, on se contente de savoir qu'un fait mince leur donne de quoi mâcher. La formation Data Science consomme cette étoile telle quelle, sans la transformer : son Cours 04 y lit les ventes par concessionnaire et par période, et c'est le même objet que celui que vous venez de bâtir.

Labo : `Labo_04_AE.md`, étape 3.

Points clés à retenir

- La couche `MARTS` est le premier endroit du projet qui s'adresse à d'autres. Elle porte des décisions, pas la forme d'une source.
- Un paquet dbt se déclare dans `packages.yml`, s'installe par `dbt deps`, et son code ne se versionne pas.
- La clé de substitution se fabrique par hachage de la clé naturelle. Déterministe, elle se recalcule dans le fait sans joindre la dimension ; en échange, rien ne garantit que la ligne existe, et c'est le test du Cours 05 qui le vérifiera.
- `date_spine` fabrique la dimension des dates, borne de fin exclue. Sa clé est la date elle-même.
- Un libellé se décode à l'endroit unique qui le produit, jamais dans le rapport qui le lit.
- Une dimension de type 1 écrase l'histoire, et il faut savoir combien cela coûte avant de livrer.
- La couche `INTERMEDIATE` porte les jointures et l'enrichissement pour que le fait reste mince. En `ephemeral`, elle n'existe pas dans la base mais elle est dans le graphe.
- Une mesure additive se stocke, un ratio jamais.
- Un fait se vérifie deux fois : son grain, par le compte des couples distincts, et son total, par un témoin qui ne vient pas du même chemin.
- Ce qu'on livre se déclare, parce que quelqu'un d'autre le lira sans pouvoir vous demander.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_04_AE.md`. Faites-les après le labo : la plupart des énoncés interrogent l'étoile que vous venez de bâtir.

© 2026 iAlexMG. Tous droits réservés.